RL for Adaptive Cache Replacement Under Changing Workloads

Lakshya Jain (B23CS1032, b23cs1032@iitj.ac.in) **Ripu Daman Singh** (B23CS1057, b23cs1057@iitj.ac.in)
Department of Computer Science and Engineering, IIT Jodhpur

GitHub: https://github.com/Damanish/adaptive_cache_replacement_rl
Interactive results: https://damanish.github.io/adaptive_cache_replacement_rl/

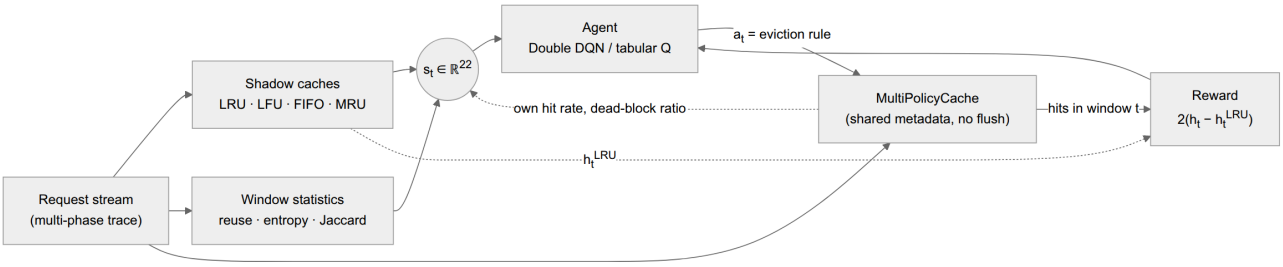


Figure 1: System overview. Once per window of $W=100$ requests the agent observes causal statistics of the past stream and picks the eviction rule used by every miss in the next window. The relative reward subtracts the action-independent shadow-LRU hit rate.

Abstract—No single cache replacement policy is best for all access patterns: LRU collapses on scans and loops, LFU on shifting working sets. We cast *policy selection* as a reinforcement-learning problem: every W requests an agent chooses which eviction rule (LRU/LFU/FIFO, optionally MRU) a shared-metadata cache applies next. On 20 held-out multi-phase traces, Double DQN beats every non-clairvoyant baseline including ARC (+0.38 pp, 18/20 traces, $p<10^{-3}$); with MRU added it reaches 63.6% hit rate (ARC 57.1%, Belady 68.6%) and cuts modelled AMAT by 19% versus LRU.

1 Problem and Related Work

Adaptive heuristics such as ARC [1], LIRS [2] and set-dueling DIP [3] hard-code *how* to adapt. Learned approaches predict per cache line, imitating Belady's MIN (Hawkeye [4], Parrot [5]) or distilling an RL agent into a hardware policy (RLR [6]). Closest to us, LeCaR [7] and CACHEUS [8] weight expert policies online by regret minimisation. We instead learn an *offline-trained, state-conditioned* selector over expert policies: one cheap inference per window instead of per request, and the agent can exploit cues (scan stride, loop reuse distance) that regret weights see only after losses accumulate.

Workloads. Traces concatenate 6 random phases (3–8k requests each, disjoint key ranges) from five regimes, each favouring a different policy: *zipf* ($\alpha \in [0.9, 1.2]$, frequency), *drift* (hot set of 0.6–0.9C that relocates every 3–6C requests, recency), *scan* (hot set plus one-shot sequential runs), *stack* (LRU stack-distance model) and *loop* (cyclic sweep over 1.1–1.5C blocks plus a small hot set). Train/val/test use disjoint seeds (64/8/20 traces); $C=W=100$.

2 RL Formulation

An episode is one trace; step t covers requests $[tW, (t+1)W)$, a semi-MDP over macro-actions.

State $\mathscr{S} \subset \mathbb{R}^{22}$ (identical layout for 3 and 4 actions; computed only from windows $< t$, verified by a causality test): (i) hit rates of four *shadow caches* running fixed LRU/LFU/FIFO/MRU on the same stream, last window and EMA ($\beta=0.8$) (8 dims); (ii) stream statistics of the last window (7 dims): unique-key ratio, sequential-stride ratio $\Pr[k_i = k_{i-1}+1]$, first-reference ratio, reuse-interval buckets $\Delta \le C$ and $C < \Delta \le 4C$, normalised popularity entropy $-\sum p_i \ln p_i / \ln W$, and key-set Jaccard overlap with the previous window (a phase-change cue); (iii) agent state (7 dims): own hit rate and its EMA, fraction of resident blocks never re-referenced, one-hot previous action (4 slots).

Action $\mathscr{A}$ (discrete): {LRU, LFU, FIFO}, or {LRU, LFU, FIFO, MRU} in the extended setting. The action fixes the victim rule for all misses in window t.

Reward $\mathscr{R}$**.** Raw hit rate swings with phase difficulty (Belady reaches only 0.29 on scans but 0.83 on drift), inflating TD-target variance. We use a relative reward

$$r_t = 2\,(h_t - h_t^{\mathrm{LRU}}) - \lambda\,\mathbf{1}[a_t \neq a_{t-1}], \tag{1}$$

where h_t^{LRU} is the shadow-LRU hit rate of the *same* window. Because the request stream is exogenous, this term is independent of the agent's actions: it shifts all action values of a state equally and leaves the greedy policy unchanged. We use $\lambda=0$; a hit/miss reward $2h_t-1$ and a latency reward $-\mathrm{AMAT}_t/L_{\mathrm{miss}}$ are also implemented.

Environment. `MultiPolicyCache` keeps last-access time, insertion time and an exponentially aged frequency $f_i \leftarrow f_i 2^{-(t-t_{\mathrm{last}})/H}+1$ ($H=10C$) for every slot, so a policy switch only changes the victim rule and never flushes the cache. Aging stops LFU from keeping blocks that were hot in an earlier phase.

Algorithms. *Tabular Q-learning* on a discretised state (best shadow policy × its margin × own-hit-rate bin × previous action × Jaccard flag; 216/384 states), $\alpha=0.1$, 300 episodes. *Double DQN* [9]: MLP $22 \to 128 \to 128 \to |\mathscr{A}|$, Huber loss, Adam ($5\times10^{-4}$), replay 50k, Polyak $\tau=0.01$, 150 episodes (≈49k steps). Both use $\gamma=0.9$, linear ε-decay $1 \to 0.03$ over 60% of training, and keep the checkpoint with the best validation hit rate (Fig. 4).

Baselines. Fixed LRU/LFU/FIFO/MRU; Random; FTL (pick last window's best shadow policy, i.e. the same information without learning); Phase-Oracle (best *single* fixed policy for each true phase, chosen in hindsight); ARC [1]; Belady's MIN [10] (offline optimum). ARC and MIN were fuzz-tested against invariants and brute force.

3 Results

Overall (Table 1). Per-trace paired comparisons (Wilcoxon signed-rank, 20 traces): with 3 actions DQN beats ARC by +0.38 pp (95% CI [0.19, 0.57], 18/20 traces, $p=8.5\times10^{-4}$), FTL

Table 1: Held-out test traces (C=100, 20 traces). AMAT with L_{hit}=1, L_{miss}=100. Post-chg.: mean hit rate over 10 windows after each phase change. Random/FTL/Phase-Oracle use 3 actions; Fixed-MRU is from the 4-action run.

Method	Hit rate	±std	AMAT	Post-chg.
Fixed-LRU	54.71	.074	45.84	53.51
Fixed-LFU (aged)	33.14	.040	67.19	33.76
Fixed-FIFO	52.27	.073	48.26	51.28
Fixed-MRU	4.32	.028	95.73	2.01
Random	50.88	.068	49.63	50.80
FTL	55.73	.075	44.83	53.41
Phase-Oracle	56.79	.067	43.77	52.54
ARC [1]	57.11	.066	43.46	54.20
Q-learning (3 act.)	56.79	.070	43.78	53.75
DQN (3 act.)	**57.50**	.068	**43.08**	**54.49**
Q-learning (+MRU)	60.09	.070	40.51	54.49
DQN (+MRU)	**63.58**	.066	**37.06**	**58.03**
Belady MIN (offline)	68.63	.072	32.06	66.19

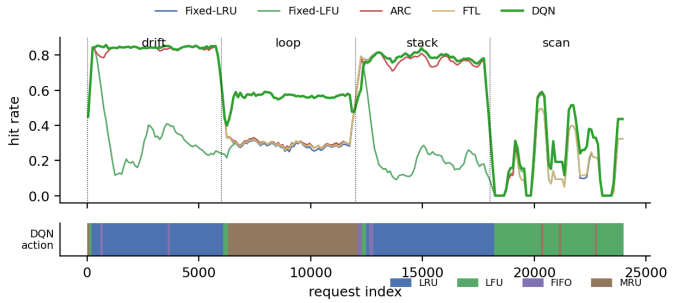


Figure 2: Drift→loop→stack→scan scenario (+MRU). DQN switches to MRU when the loop starts (phase mean 0.55 vs. 0.29–0.30 for LRU/ARC/FTL), back to LRU on stack, and to aged LFU on scans.

by +1.77 pp and LRU by +2.79 pp (20/20 each). Tabular Q-learning beats FTL and LRU but trails ARC by 0.32 pp (p=0.04). With MRU, DQN leads ARC by +6.46 pp (19/20, $p<10^{-5}$) and lowers AMAT by 19.2% versus LRU.

Where the gain comes from (Fig. 3, 2). In the 3-action setting DQN uses aged LFU in zipf, scan and loop phases (94–99% of windows) and LRU/FIFO in drift and stack, which mainly helps on scans (0.278 vs. 0.201 for LRU) and zipf (0.608 vs. 0.547). The large +MRU gain is concentrated in loops (0.636 vs. 0.299 for ARC; Belady 0.753): the agent first lets LRU/LFU refill the cache with loop blocks, then switches to MRU to pin them. Fixed MRU fails (4.3%) because it pins stale blocks from earlier phases, so this *sequence* is something no single static policy achieves. Switching *within* phases is also why DQN beats Phase-Oracle (57.50 vs. 56.79 with 3 actions, 63.58 vs. 57.22 with 4), so Phase-Oracle is *not* an upper bound on switching policies.

Recovery and transfer. Right after a phase change all methods dip (first window: DQN 0.48, LRU 0.50, ARC 0.47); DQN has the best 10-window post-change mean among online methods (Table 1). Evaluated without retraining at C=200, DQN remains best among online methods (57.6% vs. ARC 57.0%); at C=50 it falls slightly behind ARC (56.2% vs. 56.5%).

Limitations. Traces are synthetic; agents are trained from a single seed; four full-size shadow caches add 4× the cache's metadata (set sampling as in DIP [3] would reduce this); the end of a trace is a time limit but is bootstrapped as terminal; and none

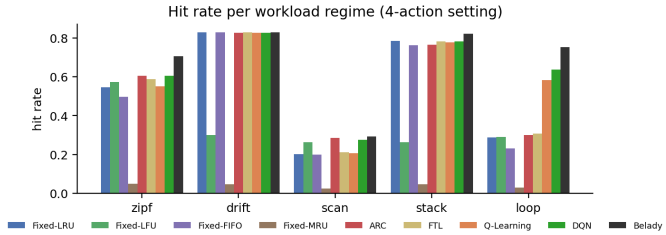


Figure 3: Hit rate per workload regime (4-action setting).

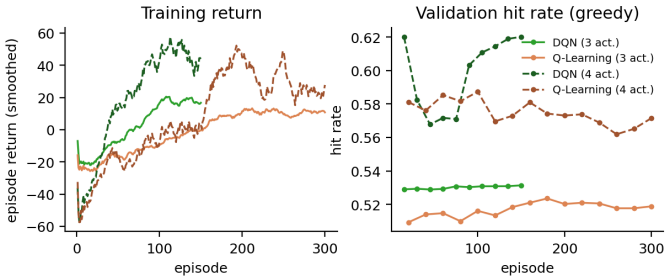


Figure 4: Training return (ε-greedy, smoothed) and greedy hit rate on the 8 validation traces. Return rises mostly as ε decays; DQN's greedy policy is near its final quality by the first evaluation (episode 15).

of our baselines defends against loops (loop-resistant LIRS [2] was not evaluated), so the +MRU gain partly reflects the larger action set.

4 Conclusion

Learning *when* to switch among simple eviction rules, rather than learning a new rule, gives a cheap (one MLP call per 100 requests), interpretable policy. It is consistently better than ARC and, with MRU available, recovers 64% of the gap between LRU and Belady's optimum.

References

[1] N. Megiddo and D. S. Modha, "ARC: A self-tuning, low overhead replacement cache," in *Proc. USENIX Conf. File and Storage Technologies (FAST)*, 2003, pp. 115–130.

[2] S. Jiang and X. Zhang, "LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance," in *Proc. ACM SIGMETRICS*, 2002, pp. 31–42.

[3] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely, and J. Emer, "Adaptive insertion policies for high performance caching," in *Proc. Int. Symp. Computer Architecture (ISCA)*, 2007, pp. 381–391.

[4] A. Jain and C. Lin, "Back to the future: Leveraging Belady's algorithm for improved cache replacement," in *Proc. Int. Symp. Computer Architecture (ISCA)*, 2016, pp. 78–89.

[5] E. Liu, M. Hashemi, K. Swersky, P. Ranganathan, and J. Ahn, "An imitation learning approach for cache replacement," in *Proc. Int. Conf. Machine Learning (ICML)*, 2020, pp. 6237–6247.

[6] S. Sethumurugan, J. Yin, and J. Sartori, "Designing a cost-effective cache replacement policy using machine learning," in *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*, 2021, pp. 291–303.

[7] G. Vietri, L. V. Rodriguez, W. A. Martinez, S. Lyons, J. Liu, R. Rangaswami, M. Zhao, and G. Narasimhan, "Driving cache replacement with ML-based LeCaR," in *Proc. USENIX Workshop Hot Topics in Storage and File Systems (HotStorage)*, 2018.

[8] L. V. Rodriguez, F. Yusuf, S. Lyons, E. Paz, R. Rangaswami, J. Liu, M. Zhao, and G. Narasimhan, "Learning cache replacement with CACHEUS," in *Proc. USENIX Conf. File and Storage Technologies (FAST)*, 2021, pp. 341–354.

[9] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artificial Intelligence*, 2016, pp. 2094–2100.

[10] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.